Go Learning Roadmap: After the Basics

A staged guide for someone who already knows basic Go syntax, `net/http` (`ListenAndServe`, `ServeMux`), reading and writing responses, and JSON encoding/decoding.

Stage 1: Fill the language gaps (1-2 weeks)

You can write handlers, but these are the things that make Go code idiomatic:

- **Structs, methods, and pointer receivers**: when to use `*T` vs `T`
- **Interfaces**: small interfaces, implicit satisfaction, `io.Reader` / `io.Writer`, `error`
- **Error handling**: `errors.New`, `fmt.Errorf` with `%w`, `errors.Is` / `errors.As`, custom error types
- **Slices and maps in depth**: length vs capacity, `append` behavior, nil vs empty
- **`defer`**, **`panic`**, **`recover`**: what they're for and what they're not for
- **Packages and modules**: `go mod init`, project layout, exported vs unexported names
- **Generics** (basic level): type parameters and constraints

Resources: A Tour of Go, *Go by Example*, and *Effective Go*.

Stage 2: Concurrency (1-2 weeks)

This is Go's signature feature, and every server you write uses it under the hood.

- Goroutines and `sync.WaitGroup`
- Channels (buffered and unbuffered), `select`
- `sync.Mutex`, `sync.RWMutex`, and when to prefer them over channels
- **`context.Context`**: cancellation, timeouts, passing request-scoped values (essential for HTTP servers)
- Common patterns: worker pools, fan-out/fan-in
- Race conditions and the `-race` flag

Stage 3: Testing (1 week, then keep doing it)

- The `testing` package, table-driven tests

- `net/http/httptest` for testing your handlers
- Benchmarks (`go test -bench`)
- Mocking through interfaces

Start writing tests for everything you build from here on. It's the fastest way to learn good design in Go.

Stage 4: Build a real REST API (2-4 weeks)

This is where most of your learning will happen. Build something like a **task manager or notes API** with:

1. CRUD endpoints using the standard library (Go 1.22+ `ServeMux` supports methods and path params like `GET /tasks/{id}`)
2. **PostgreSQL** with `database/sql` and the `pgx` driver (later try `sqlc` for generated queries)
3. Database migrations (`golang-migrate`)
4. Layered structure: handler → service → repository
5. Input validation and consistent JSON error responses
6. Middleware: logging, panic recovery, request IDs, CORS
7. **Authentication**: password hashing with bcrypt, JWT or session-based auth
8. Config through environment variables
9. Graceful shutdown using `context` and `signal.NotifyContext`
10. Structured logging with `log/slog`

Stick with the standard library first. Only after that, try a router like `chi` or a framework like Gin or Echo so you understand what they're abstracting.

Stage 5: Production skills (ongoing)

- **Docker**: write a multi-stage Dockerfile for your API
- **Docker Compose** to run your app and Postgres together
- **CI** with GitHub Actions (run `go vet`, `go test`, linting with `golangci-lint`)
- Caching with Redis
- Rate limiting
- Profiling with `pprof`

- Basic OpenAPI/Swagger documentation

Stage 6: Go further (pick based on interest)

- **gRPC and Protocol Buffers**
- **Message queues** (RabbitMQ, Kafka, NATS)
- **Building CLI tools** with `cobra`
- **Websockets** for real-time features
- **Reading open-source Go code**: the standard library itself is excellent reading

Suggested projects, in order

1. **CLI todo app** (practice structs, files, JSON)
2. **URL shortener** (HTTP, maps, then a database)
3. **Task or notes REST API with auth** (the Stage 4 project)
4. **Concurrent web scraper or link checker** (goroutines, channels, context)
5. **Chat server with websockets** (concurrency plus real-time)

Habits that speed up learning

- Run `gofmt` and `go vet` always, and read the compiler errors carefully
- Read the standard library docs at pkg.go.dev instead of relying only on tutorials
- Write small programs to test one concept at a time
- Don't reach for frameworks or heavy abstractions early; Go rewards simplicity

Recommended books and resources

- *Learning Go* by Jon Bodner (best modern book for your level)
- *Let's Go* and *Let's Go Further* by Alex Edwards (very practical for web APIs)
- *Concurrency in Go* by Katherine Cox-Buday (later, for depth)
- Gophercises and Exercism's Go track for practice problems